

Code Running Guide

How to Run and Extend the Python Examples

Installation, smoke tests, normal runs, outputs, and troubleshooting

Companion to the cyber control and PINN/PIDL lecture notes

Contents

1	What This Guide Covers	2
2	Folder Structure	2
3	Installation	2
4	Quick Smoke Test	2
5	Normal Runs	3
5.1	Classical FBSM baseline	3
5.2	DDQN defender	3
5.3	MADRL attacker-defender game	3
5.4	Inverse PINN	3
5.5	PIDL missing mechanism	3
5.6	Control PINN	3
5.7	PMP-informed PINN	3
6	How to Interpret Outputs	4
7	Extending the Code	4
7.1	From simple malware to APT	4
7.2	From compartment model to degree-based model	4
7.3	From scripted attacker to learned attacker	4
7.4	From open-loop to feedback control PINN	4
8	Troubleshooting	5
9	Reproducibility Checklist	5

1. What This Guide Covers

This guide is a standalone manual for the Python examples. It explains how to set up the environment, run smoke tests, run longer examples, interpret outputs, and extend the code to more complex cyber models.

2. Folder Structure

After unpacking the bundle, the relevant files are:

```
code/  
  requirements.txt  
  README_RUN_CODE.md  
  run_smoke_tests.sh  
  cyber_dynamics.py  
  cyber_hybrid_env.py  
  fbsm_malware_baseline.py  
  ddqn_cyber_defense.py  
  madrl_ctde_hybrid_game.py  
  inverse_pinn_sir_malware.py  
  pidl_unknown_mechanism.py  
  control_pinn_malware.py  
  pmp_informed_pinn_malware.py
```

3. Installation

Create a Python virtual environment and install dependencies:

```
1 python -m venv .venv  
2 source .venv/bin/activate # Windows: .venv\Scripts\activate  
3 pip install --upgrade pip  
4 pip install -r requirements.txt
```

The examples require only NumPy, PyTorch, and optionally Matplotlib. CUDA is optional.

4. Quick Smoke Test

Run all scripts in quick mode:

```
1 cd code  
2 ./run_smoke_tests.sh
```

A smoke test checks syntax and basic data flow. It does not train high-quality policies or high-accuracy PINNs.

5. Normal Runs

5.1 Classical FBSM baseline

```
python fbsm_malware_baseline.py --max-iter 100
```

Expected output: iteration logs, peak infection, objective value, final state, and mean control.

5.2 DDQN defender

```
python ddqn_cyber_defense.py --episodes 300
```

Expected output: training return, validation return, and exploration rate. Improvement is stochastic; run multiple seeds.

5.3 MADRL attacker-defender game

```
python madrl_ctde_hybrid_game.py --episodes 200
```

Expected output: defender and attacker episode returns. This file is a compact CTDE teaching example. For research, extend it with MAPPO clipping, GAE, opponent pools, and cross-play evaluation.

5.4 Inverse PINN

```
python inverse_pinn_sir_malware.py --iters 5000
```

Expected output: loss, learned beta, learned gamma, data loss, and residual loss.

5.5 PIDL missing mechanism

```
python pidl_unknown_mechanism.py --iters 5000
```

Expected output: loss, known-model parameter estimates, and average learned correction magnitude.

5.6 Control PINN

```
python control_pinn_malware.py --iters 5000
```

Expected output: objective proxy, state residual, and mean control.

5.7 PMP-informed PINN

```
python pmp_informed_pinn_malware.py --iters 5000
```

Expected output: state residual, costate residual, stationarity residual, and boundary residual.

6. How to Interpret Outputs

Output	Interpretation
Training return	Cumulative reward during learning. It is noisy and should not be the only metric.
Validation return	Performance of the current policy without exploration against a fixed scenario.
Peak infected	Maximum compromised population; useful cyber-risk metric.
Area under $I(t)$	Total compromise burden over time; often more informative than final infection.
Data loss	How well PINN/PIDL fits observations.
Residual loss	How well the learned trajectory satisfies the differential equation.
Stationarity loss	How well PMP control optimality is satisfied.
Correction magnitude	Whether PIDL is using the learned correction heavily or only mildly.

7. Extending the Code

7.1 From simple malware to APT

Add a deception state z and modify the infection rate:

$$\beta SI \rightarrow \beta(1 - \chi z)VC. \quad (1)$$

Add actions such as deceive, isolate, and stealth. The provided environment already contains these mechanisms.

7.2 From compartment model to degree-based model

Replace S, I, R by degree-indexed compartments. For example,

$$x = [S_1, I_1, R_1, \dots, S_K, I_K, R_K]. \quad (2)$$

The infection pressure should use the degree-weighted infected probability.

7.3 From scripted attacker to learned attacker

Start with `ddqn_cyber_defense.py`. Replace the scripted attacker with an actor network. Then use `madrl_ctde_hybrid_game.py` as the training template.

7.4 From open-loop to feedback control PINN

Change the control network input from t to $[x, t]$. Train over multiple initial states. This produces a feedback-like neural control law rather than one open-loop trajectory.

8. Troubleshooting

- If DDQN is unstable, reduce reward scale, reduce learning rate, enlarge replay buffer, and update target network less frequently.
- If MADRL cycles, freeze one agent for several updates, add opponent pools, and evaluate cross-play matrices.
- If PINN data loss is low but residual loss is high, increase residual weight or collocation points.
- If PINN residual loss is low but prediction is poor, the assumed model may be wrong or parameters may be unidentifiable.
- If PMP-informed PINN stationarity loss remains high, check control bounds and use projected/KKT residuals.
- If the correction network in PIDL dominates the known model, increase correction regularization.

9. Reproducibility Checklist

For every experiment, record:

- model equations and parameter values;
- random seed and initial states;
- action space and reward formula;
- ODE solver and step size;
- network architecture and optimizer;
- loss weights for PINN/PIDL;
- number of episodes or iterations;
- all baselines used for comparison.